Disclaimer:
This document is generated by OpenAI's ChatGPT and is meant for students to better understand padding concepts in `struct` and `union` types in C.

Sizes

- `Size of s: 24` → struct gets padding for 8-byte `double` alignment: `4 (int) + 4 (pad) + 8 (double) + 1 (char) + 7 (pad) = 24`.
- `Size of u: 8` → union size = largest member (`double`).

(Use `%zu` for `sizeof`: `printf("%zu", sizeof s);`)

Struct prints

Struct members are independent, but reading uninitialized members is **undefined behavior** (they just contain whatever bits were on the stack).

1. After `s.i = 5;`
   `[A] 5, 0.000000,`

- `s.i` is 5.
- `s.d` happens to be all-zero bits (or so tiny it rounds to 0.000000 at 6 decimals).
- `s.c` is an uninitialized byte—here it's NUL (`'\0'`), so you see "nothing".

2. After `s.d = 6.5;`
   `[B] 5, 6.500000,`

- `s.i` still 5 (separate storage).
- `s.d` now 6.5.
- `s.c` still uninitialized, apparently NUL.

3. After `s.c = 'd';`
   `[C] 5, 6.500000, d`

- Now all three are initialized.

Union prints

All union members **share the same bytes**. Only the **last written field** is meaningful; reading others just reinterprets those bytes.

1. After `u.i = 5;`
   `[A] 5, 0.000000,`

- Memory holds 4 bytes for `5` (`0x05 00 00 00` on little-endian) plus 4 leftover bytes (often zeros).
- Interpreting all 8 bytes as a `double` gives an *extremely tiny* subnormal—`%f` rounds it to `0.000000`.
- `u.c` shows the first byte `0x05` → a control char, prints as "blank".

2. After `u.d = 6.5;`
   `[B] 0, 6.500000,`

- IEEE-754 `double` for 6.5 is `0x401A000000000000`.
- The **lowest 4 bytes are zero** on little-endian, so reinterpreting them as `int` gives `0`.
- First byte is `0x00`, so `u.c` looks empty.

3. After `u.c = 'd';` (0x64)
   `[C] 100, 6.500000, d`

- You changed only the first byte to `0x64`.
- The low 4 bytes as an `int` become `0x00000064` → `100`.
- The `double`'s least-significant byte changed, but the perturbation is so tiny that `%f` still prints `6.500000` after rounding.
- `u.c` prints `d`.

🧠 What is Padding?

Padding is extra unused bytes the compiler inserts inside or at the end of a `struct` (or array of structs) to satisfy the **alignment requirements** of its members and improve performance (and in some cases, correctness).

Each data type (like `int`, `double`, `char`, etc.) usually has an **alignment requirement**: it must start at a memory address that's a multiple of some number (often its size or a factor of it).

Examples on a typical 64-bit system:

Type	Size (bytes)	Alignment (bytes)
`char`	1	1
`int`	4	4
`double`	8	8

So if a `double` must start at an address divisible by 8, and the previous field ended at an address that's not divisible by 8, the compiler will **insert padding bytes** to "skip" ahead to a valid address.

🧪 Example: Your `struct`

```
typedef struct sample_struct
{
    int i;
    double d;
    char c;
} str;
```

Let's lay this out step by step (on a 64-bit system):

Member	Type	Size	Alignment	Memory Offset	Explanation
`i`	`int`	4	4	0–3	✅ Starts at 0 (multiple of 4)
pad	—	4	—	4–7	❗ 4 bytes padding so next `double` starts at 8
`d`	`double`	8	8	8–15	✅ Starts at 8 (multiple of 8)
`c`	`char`	1	1	16	✅ Starts at 16 (alignment 1 is always OK)
pad	—	7	—	17–23	❗ Padding so the total struct size is multiple of the largest alignment (8)

🔎 Total size = 24 bytes

Why that final 7-byte padding?
If you create an array of `str`:

```
str arr[2];
```

Then `arr[0]` starts at some address `A`, and `arr[1]` must start at an address divisible by **8** (because of the `double` inside). If the total size weren't a multiple of 8, `arr[1].d` might be misaligned — so the compiler ensures the **total struct size is a multiple of the largest alignment requirement inside it**.

🧰 Padding and Performance

CPUs are fastest when data is aligned to "natural boundaries" (e.g., a 4-byte int on a 4-byte boundary). Misaligned access can be:

- **Slower** (requiring multiple memory reads)
- **Disallowed** on some hardware (causing a crash)

Padding is how compilers prevent that from happening automatically.

🧪 No Padding Example

If you reorder fields carefully, you can often reduce or eliminate padding. For example:

```
typedef struct optimized_struct {
    double d; // 8 bytes
    int i;    // 4 bytes
    char c;   // 1 byte
} opt;
```

Layout now:

Member	Size	Offset	Padding
`d`	8	0–7	—
`i`	4	8–11	—
`c`	1	12	—
pad	3	13–15	✅ Only 3 padding bytes total

✅ Total size: **16 bytes** (instead of 24!)
By placing the largest member first, we minimized padding.

📦 Alignment in Unions – The Key Idea

Even though all members of a `union` **share the same memory**, the compiler still has to ensure that **any member you might store there is properly aligned**.
So, the **alignment requirement of a union** is the **strictest alignment requirement of all its members**.

🧪 Example:

```
typedef union sample_union {
    int i;      // 4 bytes, 4-byte alignment
    double d;   // 8 bytes, 8-byte alignment
    char c;     // 1 byte, 1-byte alignment
} un;
```

Let's analyze this:

Member	Size	Alignment requirement
`int`	4	4 bytes
`double`	8	8 bytes
`char`	1	1 byte

🔎 The **largest alignment** among the members is `8` (from `double`).

- The compiler therefore ensures that the **union itself is aligned to 8 bytes**.

- This guarantees that no matter which member you use (`int`, `double`, or `char`), it will always be stored at a valid, properly aligned address.

📏 Size and Padding in a Union

The size of a union is always:

- At least the size of its **largest member**.
- Sometimes padded so that the union's **overall size** is a multiple of its alignment requirement.

In our example:

- Largest member: `double` (8 bytes)
- Required alignment: 8 bytes
- So, `sizeof(un)` will be **8** (no extra padding needed here).

If one of the members required 16-byte alignment (e.g., a SIMD vector type), the compiler would pad the union's size and alignment accordingly.

🧠 Big Difference from `struct` Padding

Feature	`struct`	`union`
Member storage	Members stored **sequentially**	All members stored **at the same location**
Padding location	May be inserted **between** members	No padding *between* members (since they overlap)
Total size	Sum of members + any padding	Size of **largest member** (+ possible final padding)
Alignment choice	Based on individual member positions	Based on **strictest** alignment among all members

---

📌 Key Takeaways

- Alignment **still applies to unions**, but only at the union's **starting address**.
- The compiler ensures that the union's memory is aligned for **any member you might store in it**.
- There is **no internal padding** (members share the same space), but the **union as a whole** might be padded to meet alignment requirements.
- The size of a union = size of the **largest member**, possibly rounded up for alignment.
